

PROGETTO TRENTO EASY START

Titolo del documento:

Analisi e Progettazione del Progetto Trento Easy Start

Document Info

Doc. Name	D2-trentoEasyStartAnalisiProgettazione
Doc. Number	D2 V1.0
Description	Documento di analisi e progettazione del progetto Trento Easy Start

INDICE

- 1. [Sommarrio](#)
- 2. [Scopo del documento](#)
- 3. [Requisiti Funzionali](#)
 - [Requisiti Funzionali Comuni](#)
 - [Requisiti Funzionali Solo per Amministratore](#)
 - [Requisiti Funzionali Solo per Utente](#)
- 4. [Use Cases](#)
 - [Use Case RF1: Login](#)
 - [Use Case RF2: Registrazione Nuovo Utente](#)
 - [Use Case RF3: Ricerca Alloggi](#)
 - [Use Case RF4: Gestione Alloggi](#)
 - [Use Case RF5: Invio Messaggio di Contatto](#)
 - [Use Case RF6: Recupero Notizie](#)
 - [Use Case RF7: Recupero Servizi](#)
 - [Use Case RF8: Gestione Chat](#)
- 5. [Analisi dei Componenti](#)
 - [Definizione dei componenti](#)
 - [Diagramma dei componenti](#)
- 6. [Diagramma delle Classi](#)
 - [Diagramma delle classi complessivo](#)
- 7. [Conclusioni](#)

Sommario

Il presente documento fornisce un'analisi dettagliata e una progettazione del sistema per il progetto "Trento Easy Start". L'obiettivo è creare una piattaforma digitale centralizzata che faciliti l'integrazione dei nuovi residenti nella città di Trento, semplificando l'accesso ai servizi comunali essenziali come trasporti, sanità ed educazione. Il documento copre i requisiti funzionali e non funzionali, i use case, l'analisi dei componenti e il diagramma delle classi utilizzando l'Unified Modeling Language (UML).

Scopo del documento

Il presente documento descrive la specifica dei requisiti di sistema del progetto "Trento Easy Start" utilizzando sia il linguaggio naturale che diagrammi formali in UML. Dopo aver definito gli obiettivi e i requisiti preliminari, il documento dettaglia i requisiti funzionali, analizza i componenti del sistema e presenta il design del sistema attraverso diagrammi dei componenti e delle classi. Questo documento serve come guida per lo sviluppo e l'implementazione della piattaforma

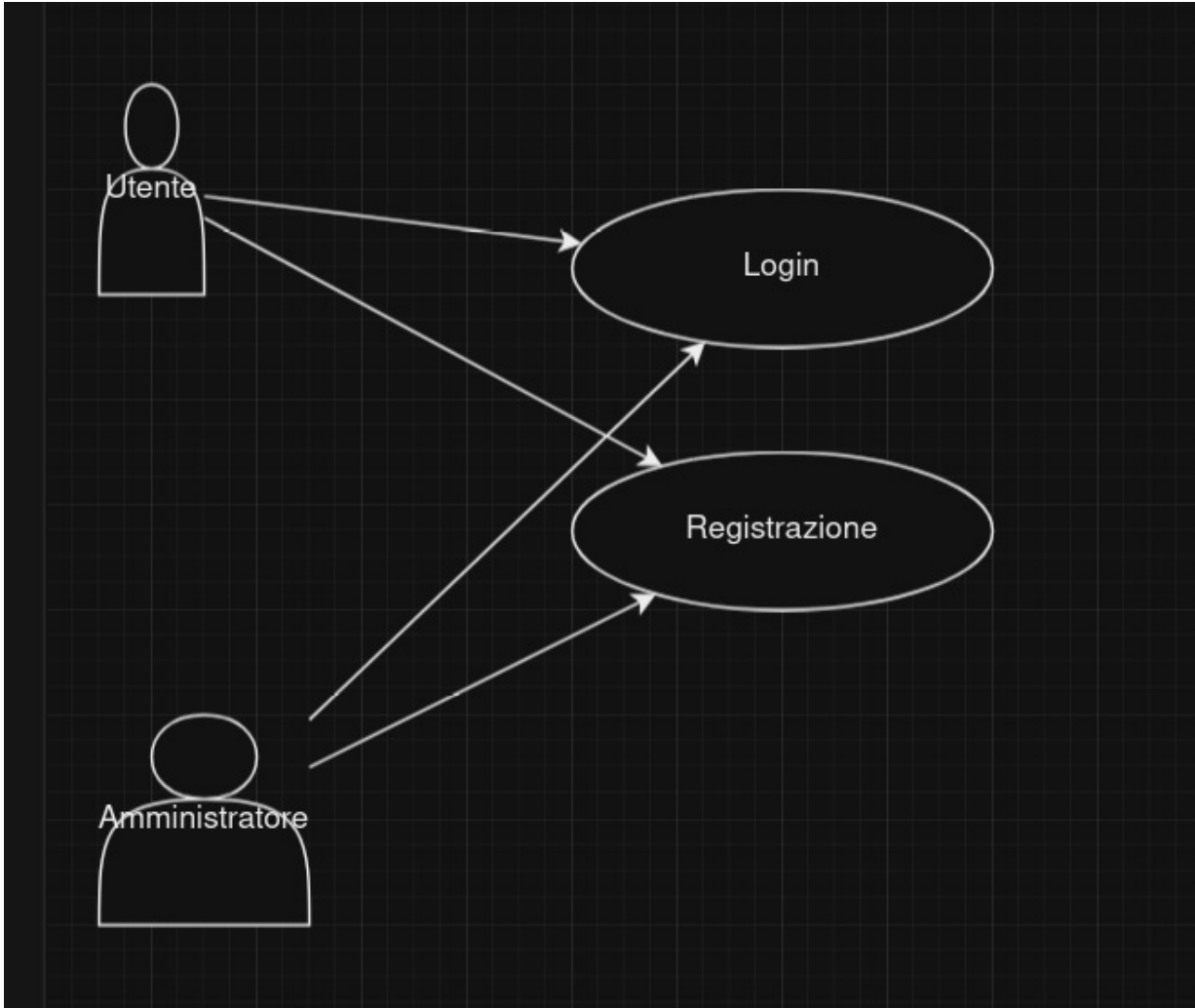
digitale, garantendo che tutte le funzionalità necessarie siano considerate e integrate efficacemente.

3. Requisiti Funzionali

Nel presente capitolo vengono elencati i requisiti funzionali del sistema, suddivisi in comuni, specifici per amministratori e specifici per utenti.

3.1 Requisiti Funzionali Comuni

RF1: Autenticazione e Autorizzazione



Riassunto:

Descrive come il sistema gestisce l'autenticazione e l'autorizzazione degli utenti e degli amministratori.

Eccezioni:

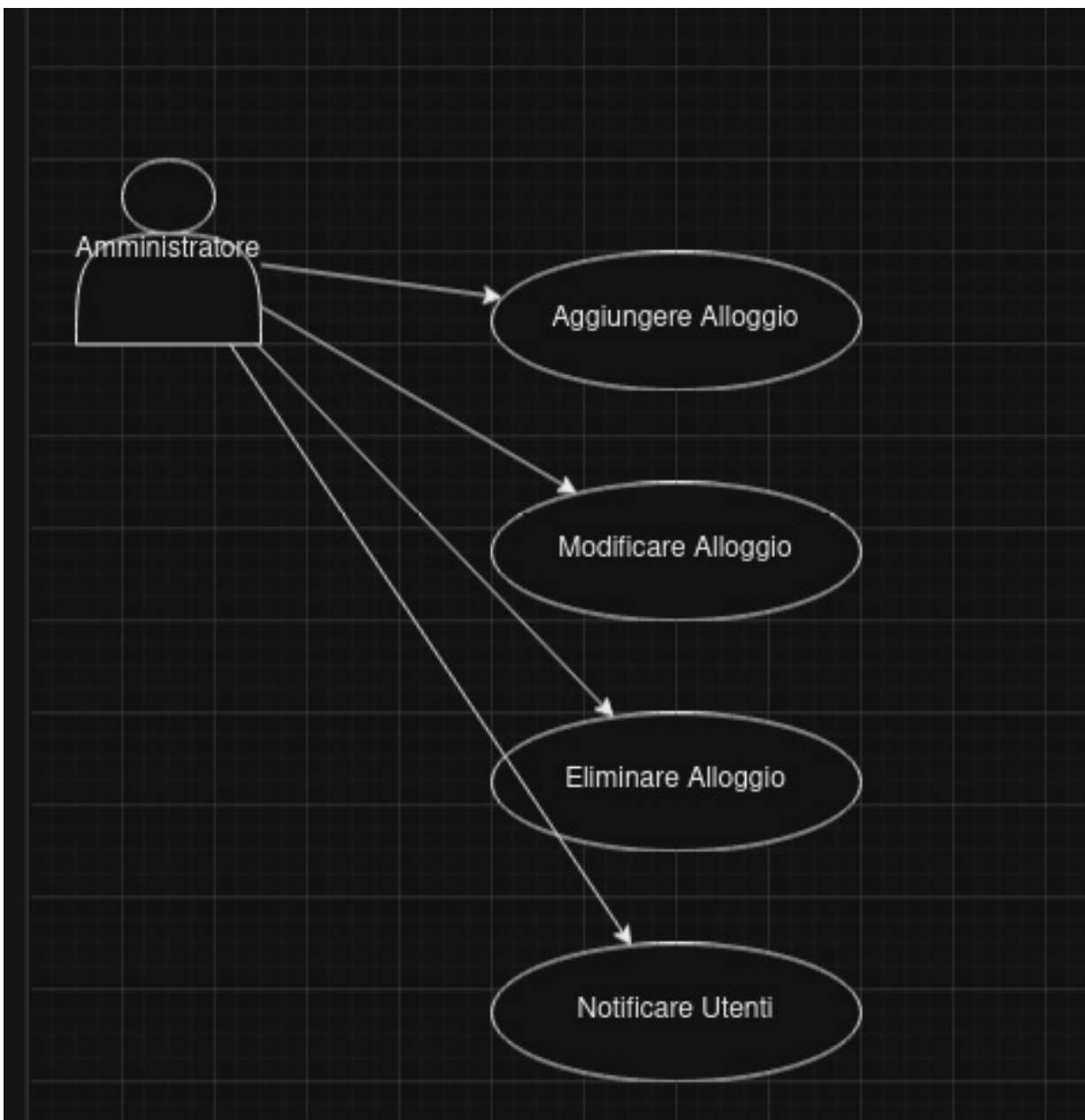
- Credenziali errate: Visualizzazione di un messaggio di errore e richiesta di ripetere il login.
- Accesso con credenziali temporanee: Obbligo di cambio password al primo accesso.
- Token scaduto o invalido: Richiesta di autenticazione nuova.

Estensioni:

- Supporto per autenticazione multi-fattore (MFA).
- Integrazione con altri provider di autenticazione (es. Facebook, LinkedIn).
- Recupero password tramite email o SMS.

3.2 Requisiti Funzionali Solo per Amministratore

RF4: Gestione Risorse

**Riassunto:**

Descrive come gli amministratori possono gestire le risorse (servizi e informazioni) all'interno della piattaforma.

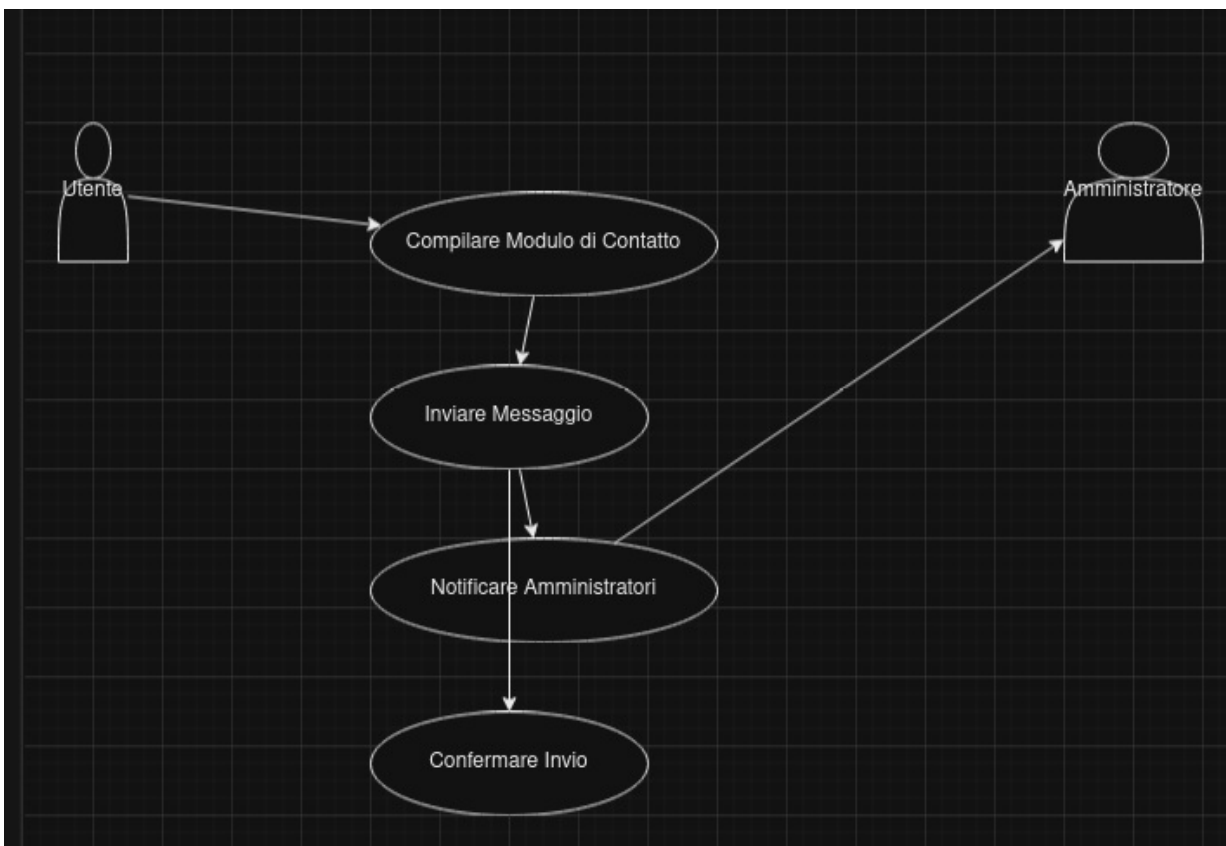
Eccezioni:

- Dati incompleti o errati: Impossibilità di salvare e visualizzazione di un messaggio di errore.
- Permessi insufficienti: Blocco dell'operazione e notifica all'amministratore.
- Errore di sistema durante l'aggiornamento: Notifica di errore e richiesta di ripetere l'operazione.

Estensioni:

- Aggiunta di categorie personalizzate per i servizi.
- Importazione ed esportazione di dati tramite file CSV o Excel.
- Notifiche automatiche agli utenti quando una risorsa viene aggiornata.

RF5: Gestione Richieste Utenti



Riassunto:

Descrive come gli amministratori possono gestire le richieste dei nuovi residenti per l'accesso a determinati servizi.

Eccezioni:

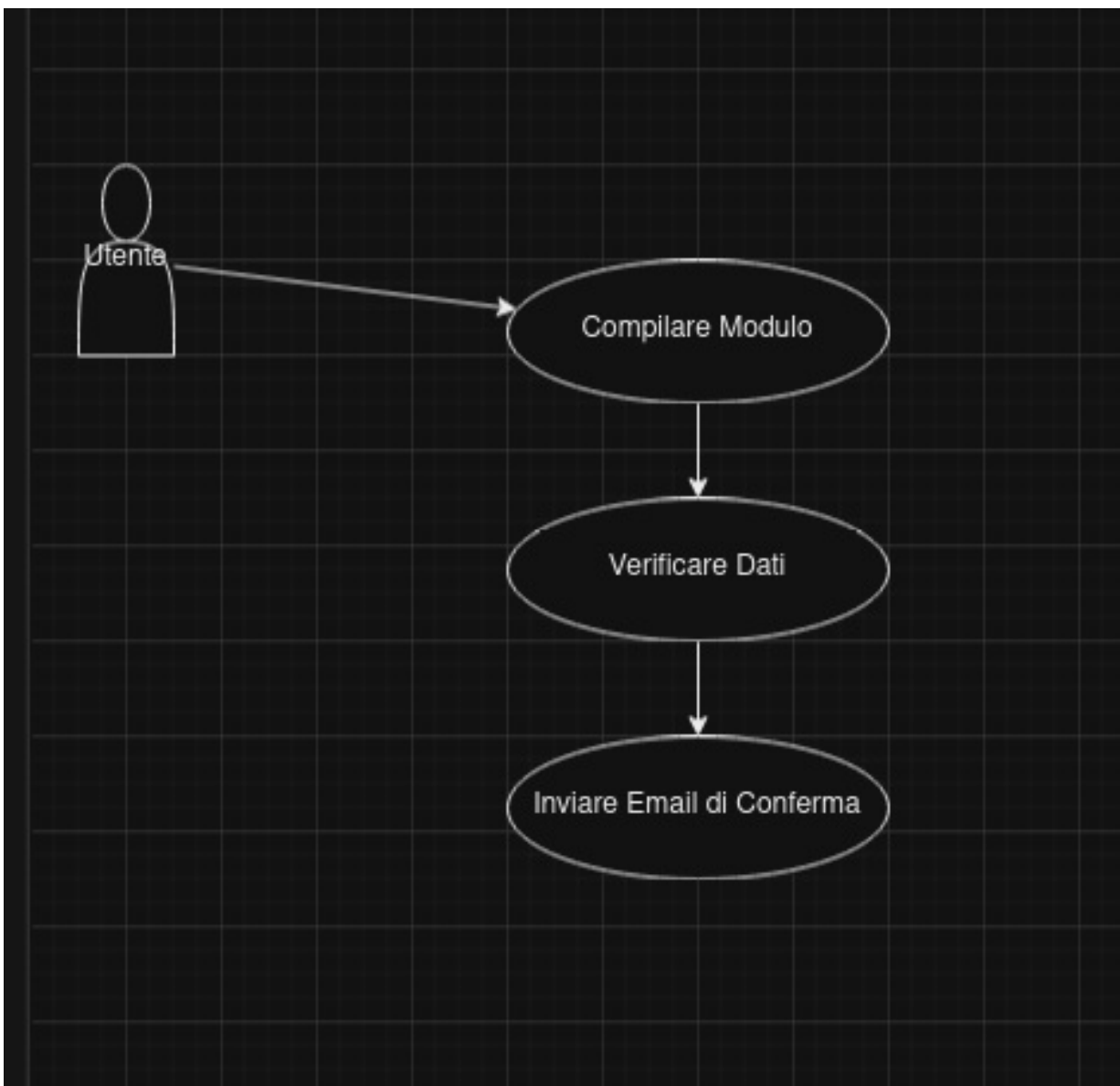
- Richiesta non conforme: Rifiuto della richiesta con motivo specificato.
- Errore durante l'approvazione o il rifiuto: Notifica di errore e richiesta di ripetere l'operazione.
- Utente non trovato: Visualizzazione di un messaggio di errore.

Estensioni:

- Sistema di feedback per gli utenti dopo la gestione della richiesta.
- Integrazione con un sistema di ticketing per tracciare le richieste.
- Aggiunta di livelli di approvazione per richieste complesse.

3.3 Requisiti Funzionali Solo per Utente

RF2: Registrazione Nuovo Utente

**Riassunto:**

Descrive come un utente anonimo effettua la registrazione nel sistema.

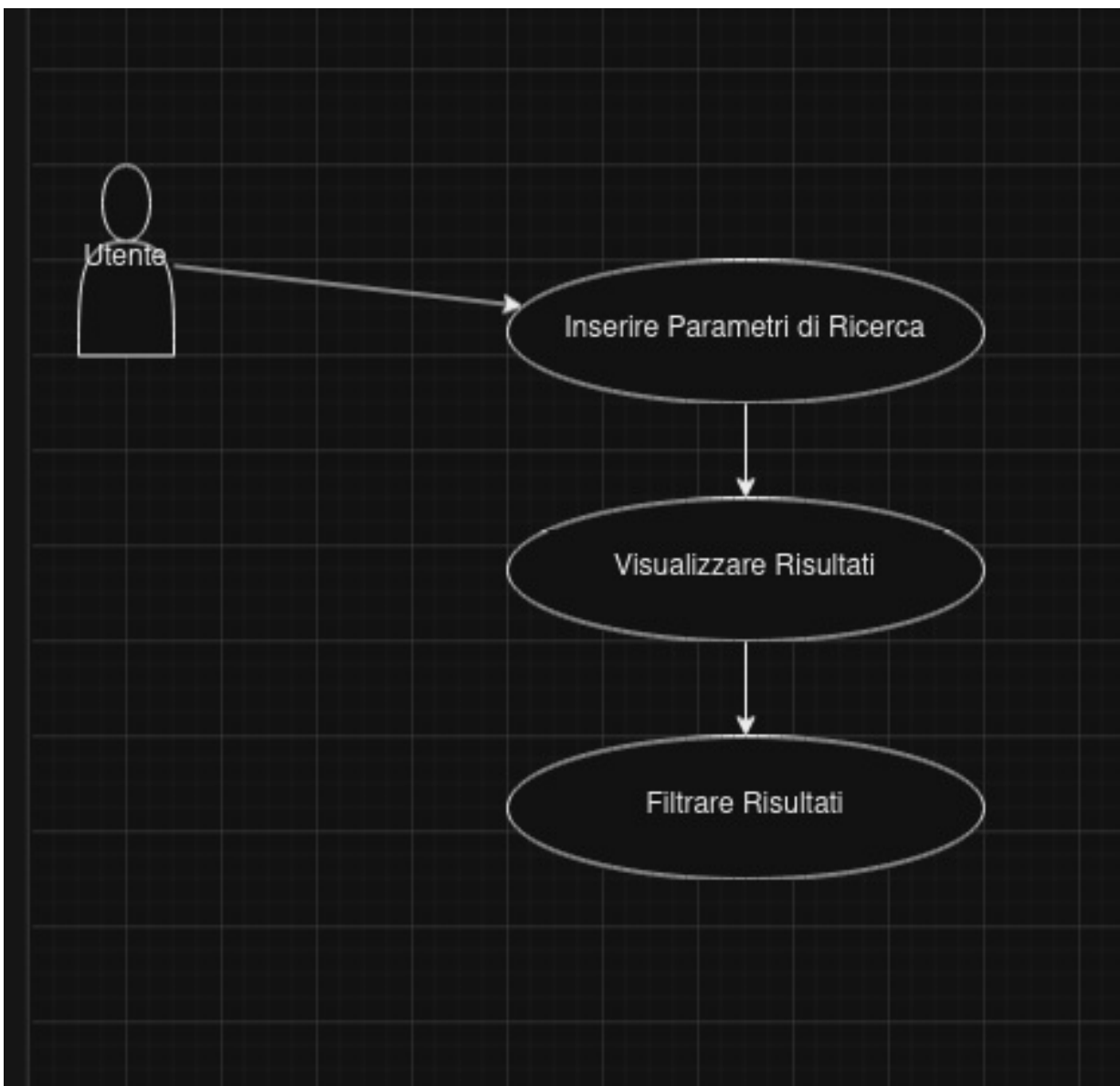
Eccezioni:

- Dati obbligatori mancanti: Registrazione non procede e viene mostrato un messaggio di errore.
- Link di conferma non cliccato: Registrazione non finalizzata.
- Email già registrata: Notifica all'utente e richiesta di effettuare il login.

Estensioni:

- Verifica del codice fiscale tramite un servizio esterno.
- Integrazione con CAPTCHA per prevenire registrazioni automatiche.
- Invio di un'email di benvenuto dopo la conferma della registrazione.

RF3: Ricerca Alloggi

**Riassunto:**

Descrive come gli utenti possono cercare alloggi disponibili nel comune di Trento e comuni limitrofi.

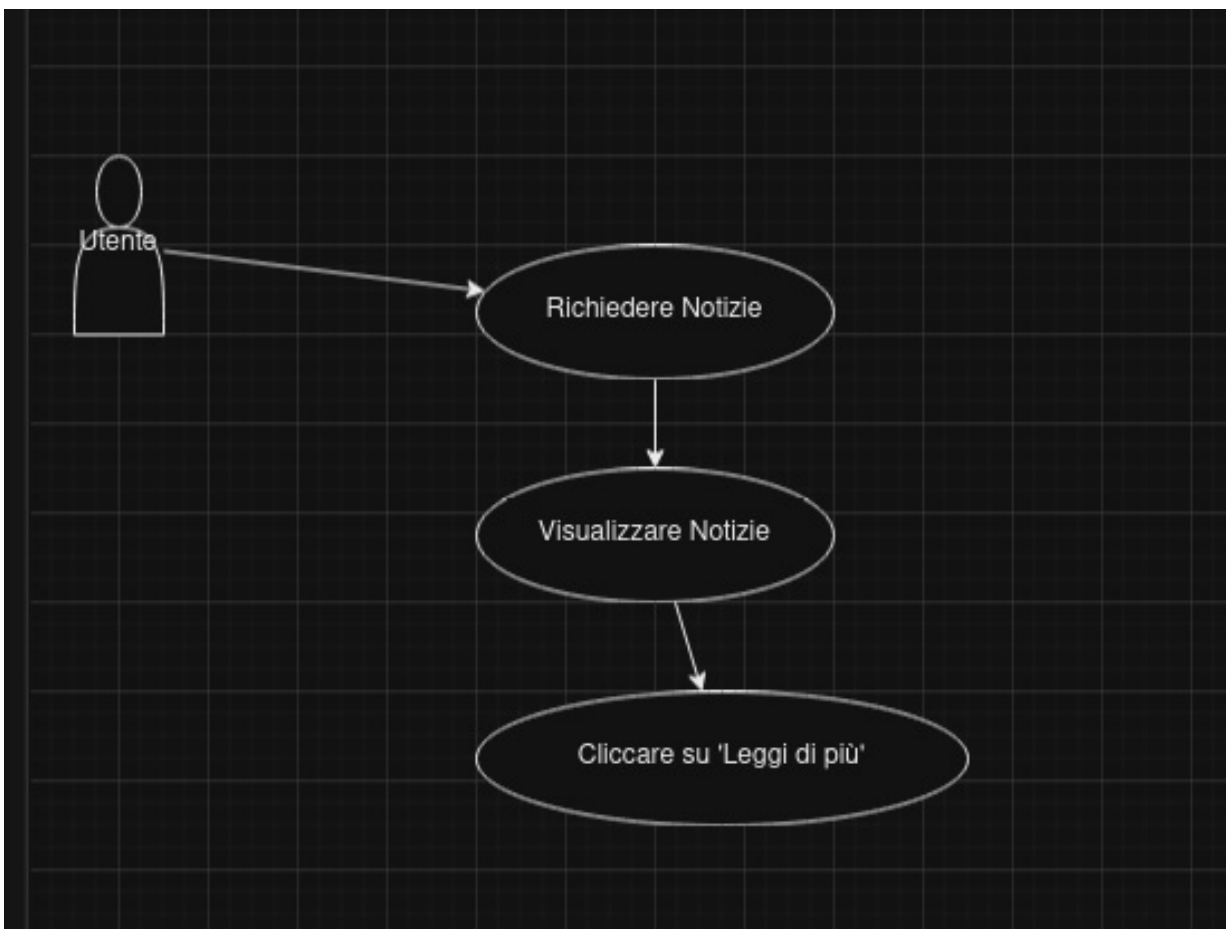
Eccezioni:

- Nessun alloggio trovato: Visualizzazione di un messaggio informativo.
- Errore nel recupero dei dati: Notifica di errore e suggerimento di ripetere la ricerca.
- Filtri non validi: Richiesta di correzione dei filtri inseriti.

Estensioni:

- Salvataggio delle ricerche preferite.
- Notifiche automatiche quando un alloggio corrispondente ai criteri viene aggiunto.
- Integrazione con mappe interattive per visualizzare la posizione degli alloggi.

RF6: Recupero Notizie

**Riassunto:**

Descrive come un utente può visualizzare le notizie recenti del Comune di Trento.

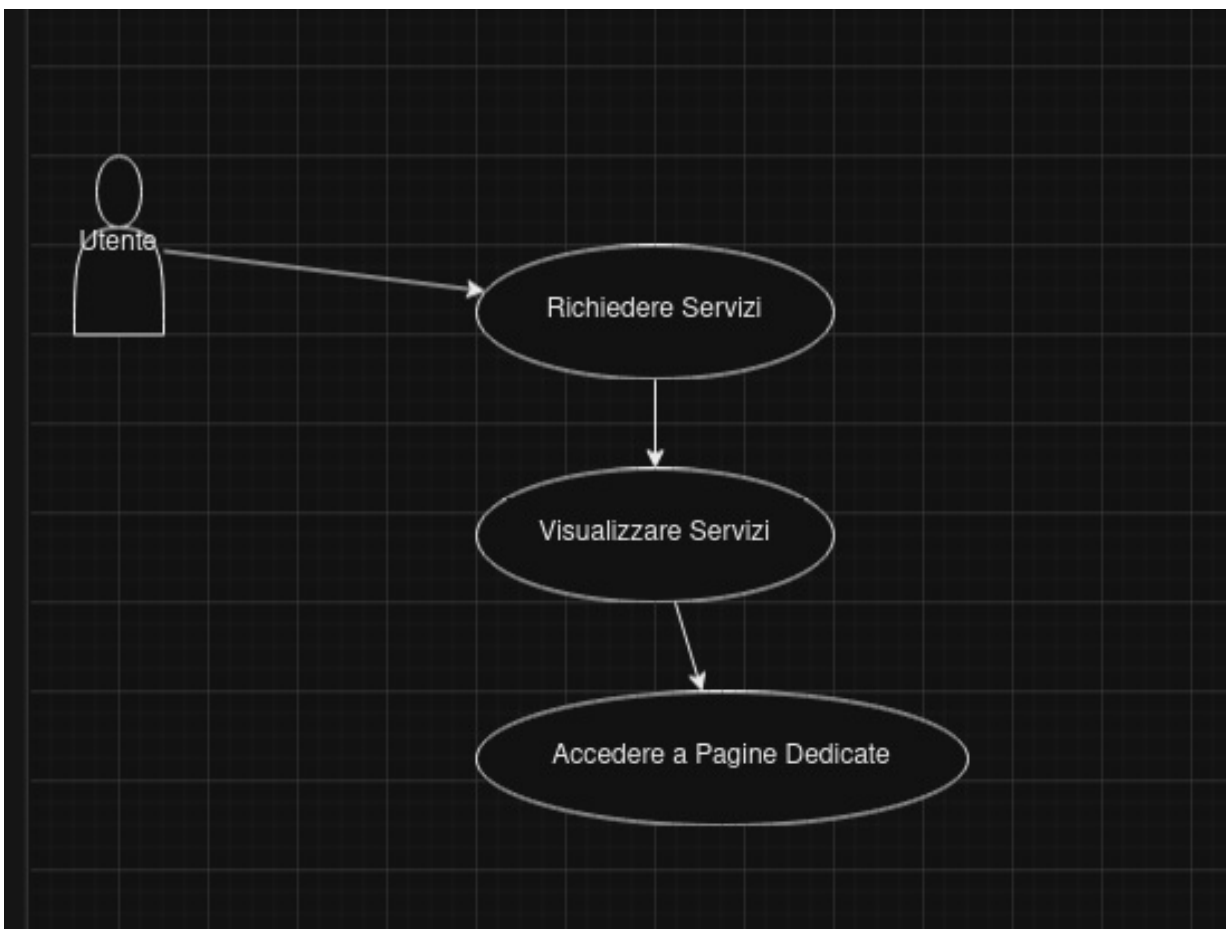
Eccezioni:

- Nessuna notizia trovata: Visualizzazione di un messaggio informativo.
- Errore nel recupero delle notizie: Notifica di errore e suggerimento di ripetere l'operazione.
- API esterna non disponibile: Notifica all'utente e visualizzazione di dati cache.

Estensioni:

- Aggiunta di filtri per categoria o data delle notizie.
- Integrazione con un sistema di commenti per le notizie.
- Possibilità di salvare le notizie preferite.

RF7: Visualizzazione Servizi

**Riassunto:**

Descrive come un utente può visualizzare i servizi offerti dal Comune di Trento.

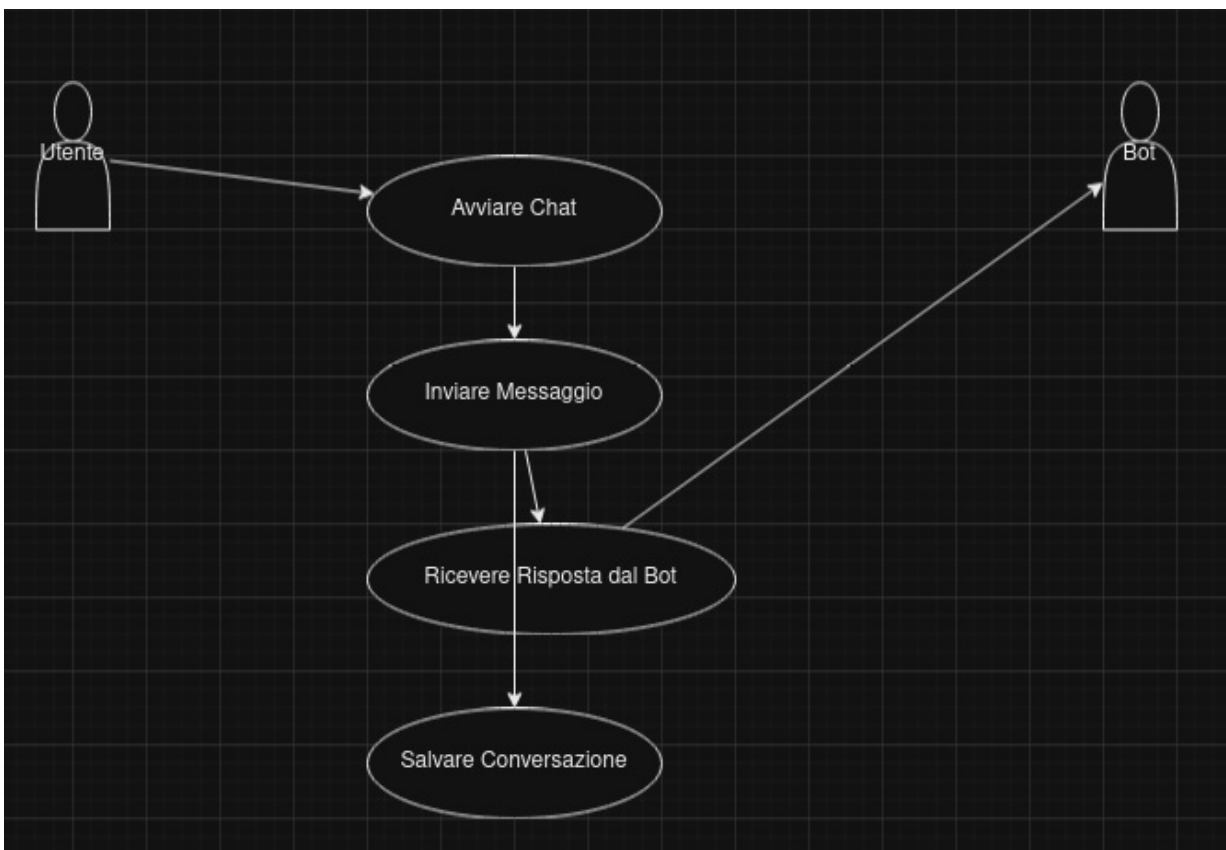
Eccezioni:

- Nessun servizio trovato: Visualizzazione di un messaggio informativo.
- Errore nel recupero dei servizi: Notifica di errore e suggerimento di ripetere l'operazione.
- API esterna non disponibile: Notifica all'utente e visualizzazione di dati cache.

Estensioni:

- Aggiunta di filtri per categoria o disponibilità dei servizi.
- Integrazione con mappe per visualizzare la localizzazione dei servizi.
- Possibilità di lasciare recensioni o feedback sui servizi utilizzati.

RF8: Gestione Chat



Riassunto:

Descrive come gli utenti possono interagire tramite la chat integrata nella piattaforma.

Eccezioni:

- Errore di connessione: Notifica di errore e suggerimento di ripetere la connessione.
- Messaggio non inviato: Notifica di errore e richiesta di ripetere l'invio.
- Bot non risponde: Visualizzazione di un messaggio informativo e possibilità di contattare un amministratore.

Estensioni:

- Aggiunta di emoticon e supporto multilingue nella chat.
- Integrazione con un sistema di ticketing per le richieste complesse.
- Possibilità di avviare chat private con gli amministratori.

4. Use Cases

In questo capitolo vengono descritti i principali use case del sistema "Trento Easy Start", suddivisi per ruolo e funzionalità.

4.1 Use Case RF1: Login

Use Case RF1: Login



Riassunto:

Descrive come un utente anonimo effettua il login nel sistema.

Eccezioni:

- Credenziali errate: Messaggio di errore e richiesta di ripetere il login.
- Prima login con password temporanea: Obbligo di cambio password.

Estensioni:

- Recupero password tramite email o SMS.
- Autenticazione multi-fattore (MFA).
- Login tramite provider esterni (es. SPID, Google).

4.2 Use Case RF2: Registrazione Nuovo Utente

Use Case RF2: Registrazione Nuovo Utente



Riassunto:

Descrive come un utente anonimo effettua la registrazione nel sistema.

Eccezioni:

- Dati obbligatori mancanti: Registrazione non procede e viene mostrato un messaggio di errore.
- Link di conferma non cliccato: Registrazione non finalizzata.
- Email già registrata: Notifica all'utente e richiesta di effettuare il login.

Estensioni:

- Verifica del codice fiscale tramite un servizio esterno.
- Integrazione con CAPTCHA per prevenire registrazioni automatiche.
- Invio di un'email di benvenuto dopo la conferma della registrazione.

4.3 Use Case RF3: Ricerca Alloggi

Use Case RF3: Ricerca Alloggi



Riassunto:

Descrive come un utente effettua la ricerca di alloggi disponibili.

Eccezioni:

- Nessun alloggio trovato: Visualizzazione di un messaggio informativo.
- Errore nel recupero dei dati: Notifica di errore e suggerimento di ripetere la ricerca.
- Filtri non validi: Richiesta di correzione dei filtri inseriti.

Estensioni:

- Salvataggio delle ricerche preferite.
- Notifiche automatiche quando un alloggio corrispondente ai criteri viene aggiunto.
- Integrazione con mappe interattive per visualizzare la posizione degli alloggi.

4.4 Use Case RF4: Gestione Alloggi

Use Case RF4: Gestione Alloggi



Riassunto:

Descrive come un amministratore può gestire le risorse alloggiative nella piattaforma.

Eccezioni:

- Dati alloggio incompleti: Impossibilità di salvare e visualizzazione di un messaggio di errore.
- Errore di sistema durante l'aggiornamento: Notifica di errore e richiesta di ripetere l'operazione.
- Permessi insufficienti: Blocco dell'operazione e notifica all'amministratore.

Estensioni:

- Aggiunta di immagini multiple tramite drag-and-drop.
- Integrazione con servizi di pagamento per prenotazioni.
- Gestione delle disponibilità in tempo reale.

4.5 Use Case RF5: Invio Messaggio di Contatto

Use Case RF5: Invio Messaggio di Contatto



Riassunto:

Descrive come un utente può inviare un messaggio di contatto al Comune di Trento tramite la piattaforma.

Eccezioni:

- Campi obbligatori mancanti: Impossibilità di inviare il messaggio e visualizzazione di un messaggio di errore.
- Errore nell'invio dell'email: Notifica di errore e suggerimento di ripetere l'operazione.
- Formato email non valido: Richiesta di correggere l'indirizzo email inserito.

Estensioni:

- Aggiunta di allegati al messaggio di contatto.
- Integrazione con un sistema di ticketing per tracciare le richieste.
- Risposte automatiche basate su parole chiave del messaggio.

4.6 Use Case RF6: Recupero Notizie

Use Case RF6: Recupero Notizie



Riassunto:

Descrive come un utente può visualizzare le notizie recenti del Comune di Trento.

Eccezioni:

- Nessuna notizia trovata: Visualizzazione di un messaggio informativo.
- Errore nel recupero delle notizie: Notifica di errore e suggerimento di ripetere l'operazione.
- API esterna non disponibile: Notifica all'utente e visualizzazione di dati cache.

Estensioni:

- Aggiunta di filtri per categoria o data delle notizie.
- Integrazione con un sistema di commenti per le notizie.
- Possibilità di salvare le notizie preferite.

4.7 Use Case RF7: Recupero Servizi

Use Case RF7: Recupero Servizi



Riassunto:

Descrive come un utente può visualizzare i servizi offerti dal Comune di Trento.

Eccezioni:

- Nessun servizio trovato: Visualizzazione di un messaggio informativo.
- Errore nel recupero dei servizi: Notifica di errore e suggerimento di ripetere l'operazione.
- API esterna non disponibile: Notifica all'utente e visualizzazione di dati cache.

Estensioni:

- Aggiunta di filtri per categoria o disponibilità dei servizi.
- Integrazione con mappe per visualizzare la localizzazione dei servizi.
- Possibilità di lasciare recensioni o feedback sui servizi utilizzati.

4.8 Use Case RF8: Gestione Chat



Riassunto:

Descrive come gli utenti possono interagire tramite la chat integrata nella piattaforma.

Eccezioni:

- Errore di connessione: Notifica di errore e suggerimento di ripetere la connessione.
- Messaggio non inviato: Notifica di errore e richiesta di ripetere l'invio.
- Bot non risponde: Visualizzazione di un messaggio informativo e possibilità di contattare un amministratore.

Estensioni:

- Aggiunta di emoticon e supporto multilingue nella chat.
- Integrazione con un sistema di ticketing per le richieste complesse.
- Possibilità di avviare chat private con gli amministratori.

5. Analisi dei Componenti

Questo capitolo presenta l'architettura del sistema "Trento Easy Start" in termini di componenti, identificando le loro responsabilità e le interfacce.

5.1 Definizione dei componenti

CMP1: Gestione Autenticazione

Descrizione: Gestisce la registrazione e l'autenticazione degli utenti e degli amministratori. Supporta l'accesso tramite credenziali locali e servizi esterni come SPID e Google.

Interfacce:

- **Richiesta:** Credenziali di accesso (email e password), autenticazione SPID/Google.
 - **Fornita:** Autenticazione riuscita, dati utente autenticato.
-

CMP2: Gestione Alloggi

Descrizione: Permette agli amministratori di gestire le risorse alloggiative e agli utenti di cercare e visualizzare gli alloggi disponibili.

Interfacce:

- **Richiesta:** Dati alloggio per creazione/modifica, parametri di ricerca.
 - **Fornita:** Elenco alloggi, dettagli alloggio.
-

CMP3: Gestione Contatti

Descrizione: Gestisce l'invio e la ricezione dei messaggi di contatto dagli utenti. Salva i messaggi nel database e invia notifiche via email agli amministratori.

Interfacce:

- **Richiesta:** Messaggi di contatto (nome, email, oggetto, messaggio).
 - **Fornita:** Conferma invio, elenco messaggi.
-

CMP4: Gestione Notizie

Descrizione: Recupera e visualizza le notizie aggiornate dal Comune di Trento. Implementa una cache per ottimizzare le performance.

Interfacce:

- **Richiesta:** Richiesta di notizie.
 - **Fornita:** Elenco notizie filtrate.
-

CMP5: Gestione Servizi

Descrizione: Recupera e visualizza i servizi offerti dal Comune di Trento, suddivisi per categoria.

Interfacce:

- **Richiesta:** Richiesta di servizi.
 - **Fornita:** Elenco servizi.
-

CMP6: Gestione Chat

Descrizione: Implementa la funzionalità di chat in tempo reale tra gli utenti e un bot integrato. Gestisce le connessioni Socket.io e l'interazione con il bot tramite l'API di Cohere.

Interfacce:

- **Richiesta:** Messaggi da utenti.
 - **Fornita:** Messaggi in tempo reale, risposte del bot.
-

CMP7: Gestione Eventi

Descrizione: Recupera e visualizza gli eventi pubblicati dal Comune di Trento, implementando una cache per migliorare le performance.

Interfacce:

- **Richiesta:** Richiesta di eventi.
 - **Fornita:** Elenco eventi futuri.
-

CMP8: Gestione Database

Descrizione: Gestisce tutte le operazioni di lettura e scrittura nel database MongoDB, inclusi utenti, alloggi, contatti, messaggi e altri dati.

Interfacce:

- **Richiesta:** Query di lettura/scrittura.
 - **Fornita:** Dati richiesti, conferme di operazioni.
-

CMP9: Gestione Invio Email

Descrizione: Gestisce l'invio di email di notifica agli utenti e agli amministratori, utilizzando Nodemailer.

Interfacce:

- **Richiesta:** Contenuti delle email (notifiche, conferme).
 - **Fornita:** Invio email riuscito o fallito.
-

CMP10: Gestione Notifiche Telegram

Descrizione: Invia notifiche al canale o gruppo Telegram specificato utilizzando l'API di Telegram, informando su eventi come nuove registrazioni o logins.

Interfacce:

- **Richiesta:** Messaggi da inviare a Telegram.
- **Fornita:** Conferma invio messaggio.

5.2 Diagramma dei componenti

Di seguito viene presentata una descrizione testuale del diagramma dei componenti del sistema "Trento Easy Start":

- **Gestione Autenticazione** interagisce con **Gestione Database** per verificare e salvare i dati degli utenti.
- **Gestione Alloggi** comunica con **Gestione Database** per operazioni CRUD sugli alloggi.
- **Gestione Contatti** interagisce con **Gestione Database** e **Gestione Invio Email** per salvare e notificare i messaggi.
- **Gestione Notizie** e **Gestione Eventi** recuperano dati tramite API esterne e li forniscono agli utenti.
- **Gestione Servizi** interagisce con API esterne per recuperare informazioni sui servizi.
- **Gestione Chat** utilizza **Gestione Database** per salvare le conversazioni e **Gestione Invio Email** per notifiche.
- **Gestione Invio Email** si interfaccia con servizi esterni per l'invio di email.
- **Gestione Notifiche Telegram** interagisce con il servizio Telegram per inviare notifiche.



6. Diagramma delle Classi

In questo capitolo vengono presentate le classi del sistema "Trento Easy Start", con i loro attributi e metodi principali, nonché le relazioni tra di esse.

6.1 Diagramma delle classi complessivo

Classe: User

Descrizione: Rappresenta un utente registrato nella piattaforma, sia residente che amministratore.

Attributi:

- id: String (ID univoco)
- name: String
- email: String (unica)
- password: String (cifrata)
- role: String (es. "utente", "amministratore")
- dateRegistered: Date

Metodi:

- register(): Registra un nuovo utente.
- login(): Effettua il login dell'utente.

- updateProfile(): Aggiorna i dati del profilo utente.
-

Classe: Accommodation

Descrizione: Rappresenta un alloggio disponibile per i nuovi residenti.

Attributi:

- id: String (ID univoco)
- title: String
- type: String (Affitti Brevi, Affitti Lunghi, Case Vacanza)
- description: String
- location: String
- price: Number
- images: Array di String
- datePosted: Date
- postedBy: ObjectId (riferimento a User)

Metodi:

- create(): Crea un nuovo alloggio.
 - update(): Aggiorna un alloggio esistente.
 - delete(): Elimina un alloggio.
 - search(): Ricerca alloggi basata su criteri.
-

Classe: Contact

Descrizione: Rappresenta un messaggio di contatto inviato da un utente.

Attributi:

- id: String (ID univoco)
- name: String
- email: String
- subject: String
- message: String
- dateSent: Date

Metodi:

- send(): Invia un messaggio di contatto.
 - retrieve(): Recupera i messaggi di contatto.
-

Classe: News

Descrizione: Rappresenta una notizia recuperata dall'API del Comune di Trento.

Attributi:

- id: String (ID univoco)
- title: String
- description: String
- date: Date
- link: String
- type: String (es. "avviso")

Metodi:

- fetch(): Recupera le notizie dall'API esterna.
 - display(): Visualizza le notizie nell'interfaccia utente.
-

Classe: Service

Descrizione: Rappresenta un servizio offerto dal Comune di Trento.

Attributi:

- id: String (ID univoco)
- title: String
- description: String
- category: String (Trasporti, Sanità, Educazione)
- link: String

Metodi:

- fetch(): Recupera i servizi dall'API esterna.
 - display(): Visualizza i servizi nell'interfaccia utente.
-

Classe: ChatMessage

Descrizione: Rappresenta un messaggio scambiato nella chat tra utenti e il bot.

Attributi:

- id: String (ID univoco)
- user: String (nome utente o 'bot')
- text: String
- date: Date

Metodi:

- sendMessage(): Invia un messaggio nella chat.
 - receiveMessage(): Riceve un messaggio dalla chat.
-

Classe: EmailService

Descrizione: Gestisce l'invio di email di notifica agli utenti e agli amministratori.

Attributi:

- smtpConfig: Object (configurazione SMTP)

Metodi:

- sendEmail(to, subject, body): Invia un'email.
 - sendRegistrationEmail(user): Invia email di registrazione.
 - sendNotificationEmail(message): Invia notifiche generiche.
-

Classe: TelegramService

Descrizione: Gestisce l'invio di notifiche al canale o gruppo Telegram.

Attributi:

- botToken: String
- chatId: String

Metodi:

- sendMessage(message): Invia un messaggio a Telegram.
-

Classe: Database

Descrizione: Rappresenta il database MongoDB utilizzato per memorizzare tutti i dati del sistema.

Attributi:

- connectionString: String

Metodi:

- connect(): Connette al database.
- disconnect(): Disconnette dal database.
- query(collection, criteria): Esegue una query sul database.
- save(collection, data): Salva dati nel database.

Classe: ExternalAPI

Descrizione: Rappresenta le interazioni con le API esterne del Comune di Trento per recuperare notizie, eventi e servizi.

Attributi:

- baseUrl: String

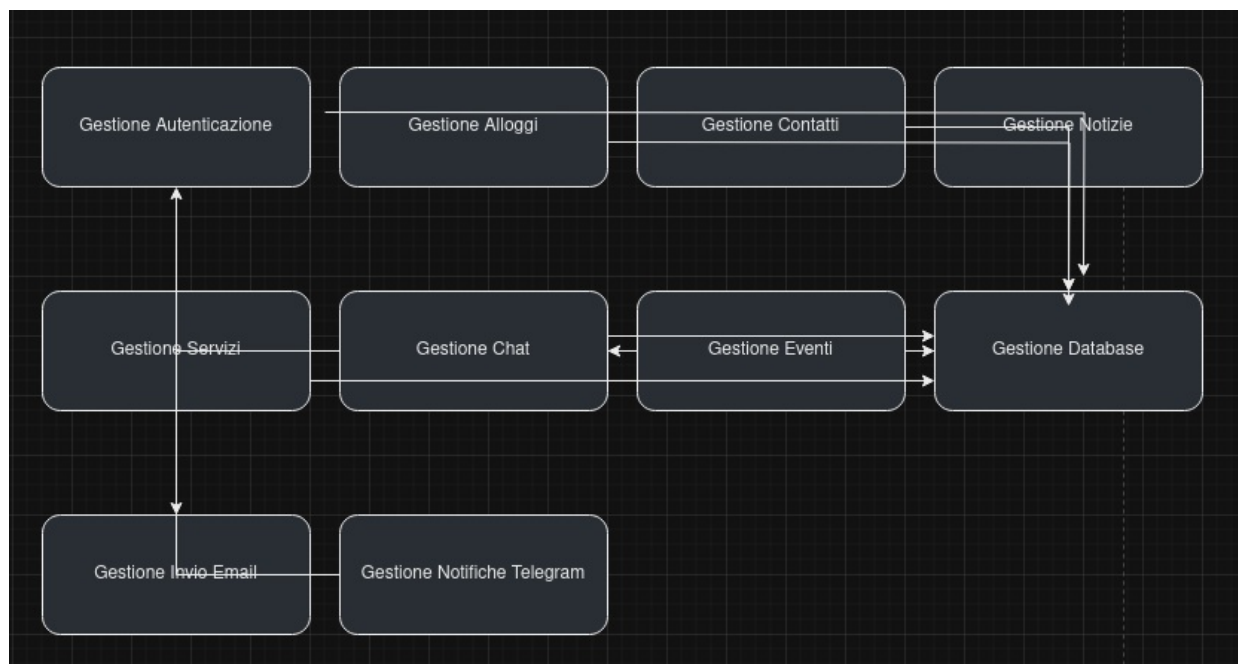
Metodi:

- fetchData(endpoint, params): Recupera dati da un endpoint specifico.
- cacheData(data, duration): Implementa la cache per ottimizzare le richieste.

5.2 Diagramma dei componenti

Di seguito viene presentata una descrizione testuale del diagramma dei componenti del sistema "Trento Easy Start":

- **Gestione Autenticazione** interagisce con **Gestione Database** per verificare e salvare i dati degli utenti.
- **Gestione Alloggi** comunica con **Gestione Database** per operazioni CRUD sugli alloggi.
- **Gestione Contatti** interagisce con **Gestione Database** e **Gestione Invio Email** per salvare e notificare i messaggi.
- **Gestione Notizie** e **Gestione Eventi** recuperano dati tramite API esterne e li forniscono agli utenti.
- **Gestione Servizi** interagisce con API esterne per recuperare informazioni sui servizi.
- **Gestione Chat** utilizza **Gestione Database** per salvare le conversazioni e **Gestione Invio Email** per notifiche.
- **Gestione Invio Email** si interfaccia con servizi esterni per l'invio di email.
- **Gestione Notifiche Telegram** interagisce con il servizio Telegram per inviare notifiche.

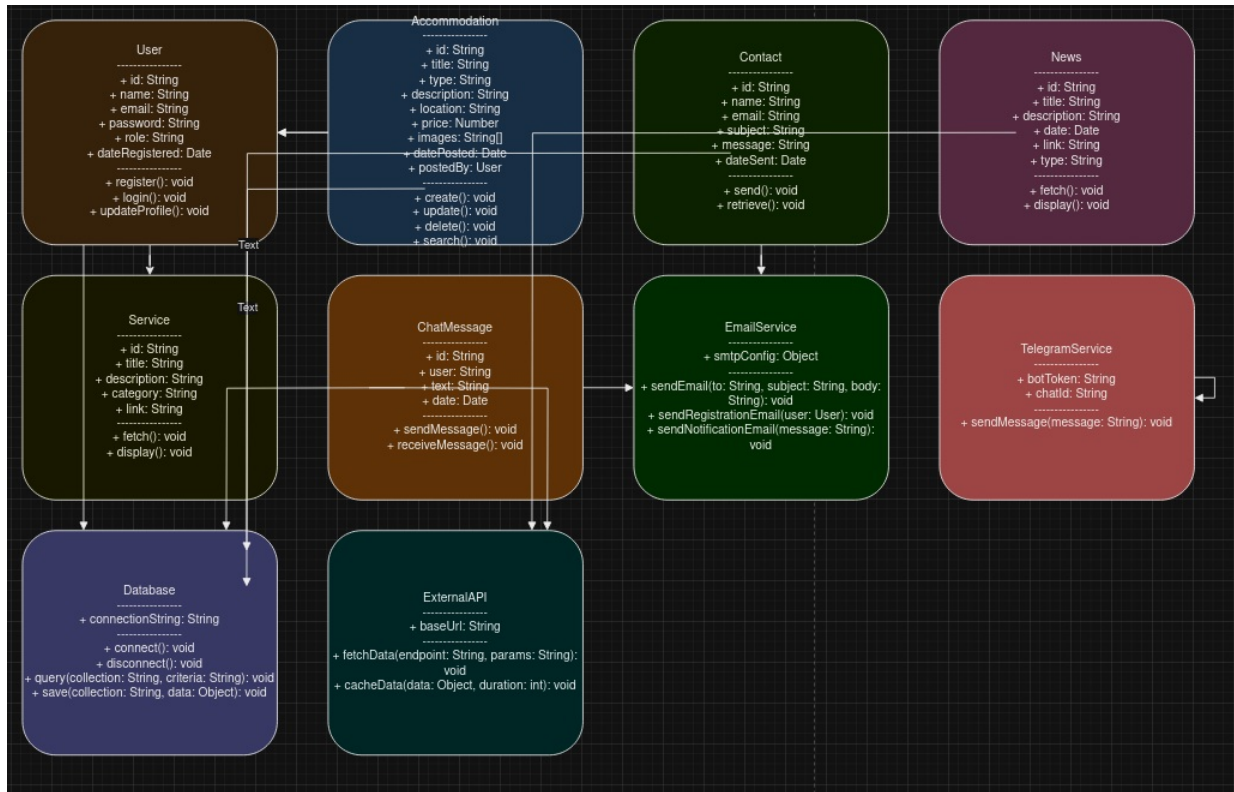


6. Diagramma delle Classi

In questo capitolo vengono presentate le classi del sistema "Trento Easy Start", con i loro attributi e metodi principali,

nonché le relazioni tra di esse.

6.1 Diagramma delle classi complessivo



Classe: User

Descrizione: Rappresenta un utente registrato nella piattaforma, sia residente che amministratore.

Attributi:

- id: String (ID univoco)
- name: String
- email: String (unica)
- password: String (cifrata)
- role: String (es. "utente", "amministratore")
- dateRegistered: Date

Metodi:

- register(): Registra un nuovo utente.
- login(): Effettua il login dell'utente.
- updateProfile(): Aggiorna i dati del profilo utente.

Classe: Accommodation

Descrizione: Rappresenta un alloggio disponibile per i nuovi residenti.

Attributi:

- id: String (ID univoco)
- title: String
- type: String (Affitti Brevi, Affitti Lunghi, Case Vacanza)
- description: String
- location: String
- price: Number
- images: Array di String
- datePosted: Date

- postedBy: ObjectId (riferimento a User)

Metodi:

- create(): Crea un nuovo alloggio.
 - update(): Aggiorna un alloggio esistente.
 - delete(): Elimina un alloggio.
 - search(): Ricerca alloggi basata su criteri.
-

Classe: Contact

Descrizione: Rappresenta un messaggio di contatto inviato da un utente.

Attributi:

- id: String (ID univoco)
- name: String
- email: String
- subject: String
- message: String
- dateSent: Date

Metodi:

- send(): Invia un messaggio di contatto.
 - retrieve(): Recupera i messaggi di contatto.
-

Classe: News

Descrizione: Rappresenta una notizia recuperata dall'API del Comune di Trento.

Attributi:

- id: String (ID univoco)
- title: String
- description: String
- date: Date
- link: String
- type: String (es. "avviso")

Metodi:

- fetch(): Recupera le notizie dall'API esterna.
 - display(): Visualizza le notizie nell'interfaccia utente.
-

Classe: Service

Descrizione: Rappresenta un servizio offerto dal Comune di Trento.

Attributi:

- id: String (ID univoco)
- title: String
- description: String
- category: String (Trasporti, Sanità, Educazione)
- link: String

Metodi:

- fetch(): Recupera i servizi dall'API esterna.
 - display(): Visualizza i servizi nell'interfaccia utente.
-

Classe: ChatMessage

Descrizione: Rappresenta un messaggio scambiato nella chat tra utenti e il bot.

Attributi:

- id: String (ID univoco)
- user: String (nome utente o 'bot')
- text: String
- date: Date

Metodi:

- sendMessage(): Invia un messaggio nella chat.
 - receiveMessage(): Riceve un messaggio dalla chat.
-

Classe: EmailService

Descrizione: Gestisce l'invio di email di notifica agli utenti e agli amministratori.

Attributi:

- smtpConfig: Object (configurazione SMTP)

Metodi:

- sendEmail(to, subject, body): Invia un'email.
 - sendRegistrationEmail(user): Invia email di registrazione.
 - sendNotificationEmail(message): Invia notifiche generiche.
-

Classe: TelegramService

Descrizione: Gestisce l'invio di notifiche al canale o gruppo Telegram.

Attributi:

- botToken: String
- chatId: String

Metodi:

- sendMessage(message): Invia un messaggio a Telegram.
-

Classe: Database

Descrizione: Rappresenta il database MongoDB utilizzato per memorizzare tutti i dati del sistema.

Attributi:

- connectionString: String

Metodi:

- connect(): Connette al database.
 - disconnect(): Disconnette dal database.
 - query(collection, criteria): Esegue una query sul database.
 - save(collection, data): Salva dati nel database.
-

Classe: ExternalAPI

Descrizione: Rappresenta le interazioni con le API esterne del Comune di Trento per recuperare notizie, eventi e servizi.

Attributi:

- baseUrl: String

Metodi:

- fetchData(endpoint, params): Recupera dati da un endpoint specifico.
- cacheData(data, duration): Implementa la cache per ottimizzare le richieste.